A4

Friday 4 April 2025
last class of term!

Plan for today:
- a few more words about rates of convergence
- a few words about the exam + office hours
- a few words on divergence
- a few words on software + algorithms for rootfinding

Last time we were describing the rates of convergence for a sequence. We had a sequence

$$x_0, x_1, x_2, \ldots \quad \text{that converges to } x^*$$

and define $e_i = |x_i - x^*|$ (the absolute error in an approximation)

We defined the rate of convergence r as the largest value r such that

$$\lim_{x_i \to x^*} \frac{e_{i+1}}{e_i^r} = \text{Constant} \neq 0.$$

bisection method : $r = 1$ linear conv.

secant method : $r \doteq 1.618$ } super-linear conv.

Newton method : $r = 2$ }

When they converge!

We saw that a super linear rate really speeds up convergence!

What if you only have data? How can you estimate the convergence rate?

How to determine convergence rates from observed data?

Suppose you have a table of observations from a sequence.

(see flip side of rates graph)

i	x_i	Δx_i
0	—	
1	—	—
2	—	—
3	—	—

How can you estimate the observed rate of convergence?

We know $\lim_{i \to \infty} \dfrac{e_{i+1}}{e_i^r} = c \neq 0$

So for sufficiently large i $\quad \dfrac{e_{i+1}}{e_i^r} \doteq \dfrac{e_i}{e_{i-1}^r}$

or $\left(\dfrac{e_{i+1}}{e_i}\right) \doteq \left(\dfrac{e_i}{e_{i-1}}\right)^r$

$\therefore \log(e_{i+1}/e_i) \doteq r \log(e_i/e_{i-1})$.

or $r = \log(e_{i+1}/e_i) / \log(e_i/e_{i-1})$.

With observations only, we don't know e_i (since it depends on knowing x^*). We can <u>estimate</u> e_i by Δx_i (assumes each step gets you to root)
$= |x_i - x_{i-1}|$
and use this to estimate r.

With this approach

$$r_{est} = \log(\Delta x_{i+1}/\Delta x_i) / \log(\Delta x_i/\Delta x_{i-1})$$

we can then produce

$$C_{est} = \frac{\Delta x_{i+1}}{\Delta x_i^{r_{est}}}$$

The results in the output can be analyzed using this approach, and we can conclude that

$$r \doteq 1.15 \qquad \text{with } c \approx 5 \times 10^{-1}$$

(the process suffers from a lack of precision as i increases, so the estimate process breaks down a little.)

Estimating a convergence rate from computational results

i	x(i)	| x(i)-x(i-1) |	rate est.	const. est
1	2.75000000000000e+00			
2	2.35916206282344e+00	3.91e-01		
3	2.15401181187523e+00	2.05e-01	-0.00e+00	2.05e-01
4	2.05816438480827e+00	9.58e-02	1.18e+00	6.22e-01
5	2.01898128955147e+00	3.92e-02	1.18e+00	6.17e-01
6	2.00523654181316e+00	1.37e-02	1.17e+00	6.11e-01
7	2.00119088895221e+00	4.05e-03	1.17e+00	6.03e-01
8	2.00021688122653e+00	9.74e-04	1.16e+00	5.95e-01
9	2.00003059322488e+00	1.86e-04	1.16e+00	5.87e-01
10	2.00000321691105e+00	2.74e-05	1.16e+00	5.77e-01
11	2.00000024128199e+00	2.98e-06	1.16e+00	5.67e-01
12	2.00000001227082e+00	2.29e-07	1.16e+00	5.57e-01
13	2.00000000039919e+00	1.19e-08	1.15e+00	5.47e-01
14	2.00000000000777e+00	3.91e-10	1.15e+00	5.37e-01
15	2.00000000000008e+00	7.68e-12	1.15e+00	5.28e-01
16	2.00000000000000e+00	8.35e-14	1.15e+00	5.11e-01
17	2.00000000000000e+00	4.44e-16	1.16e+00	6.19e-01
18	2.00000000000000e+00	0.00e+00	Inf	NaN
19	2.00000000000000e+00	0.00e+00	NaN	NaN
20	2.00000000000000e+00	0.00e+00	NaN	NaN

Course wrap up:

- see Exam Info page on Quercus
  - exam period office hours
  - will post a practice exam next week
  - exam aids:
    - calculator
      - show some steps
    - double sided, handwritten $8\frac{1}{2}'' \times 11''$ aid sheet

- will post sample solutions to Assignments and will let you know when marked work available.
- will complete remark requests
- the exam will cover material from throughout the course.

- will post coverpage of exam

We have seen that Newton's method and the secant method converge faster than the bisection method. So why not always use them?

<u>example</u> Apply Newton's method to find a root of $f(x) = x - \tan(x)$ in $[3.5, 4.5]$

Since $f(3.5) \doteq +3.13$ and $f(4.5) \doteq -0.137$ and $f(x)$ is continuous in this interval, by the IVT there is a root in the interval.

$$
\begin{aligned}
f'(x) &= 1 - \sec^2(x) \\
&= 1 - \frac{1}{\cos^2(x)} \\
&= \frac{\cos^2(x) - 1}{\cos^2(x)} \\
&= -\frac{\sin^2(x)}{\cos^2(x)} \\
&= -\tan^2(x)
\end{aligned}
$$

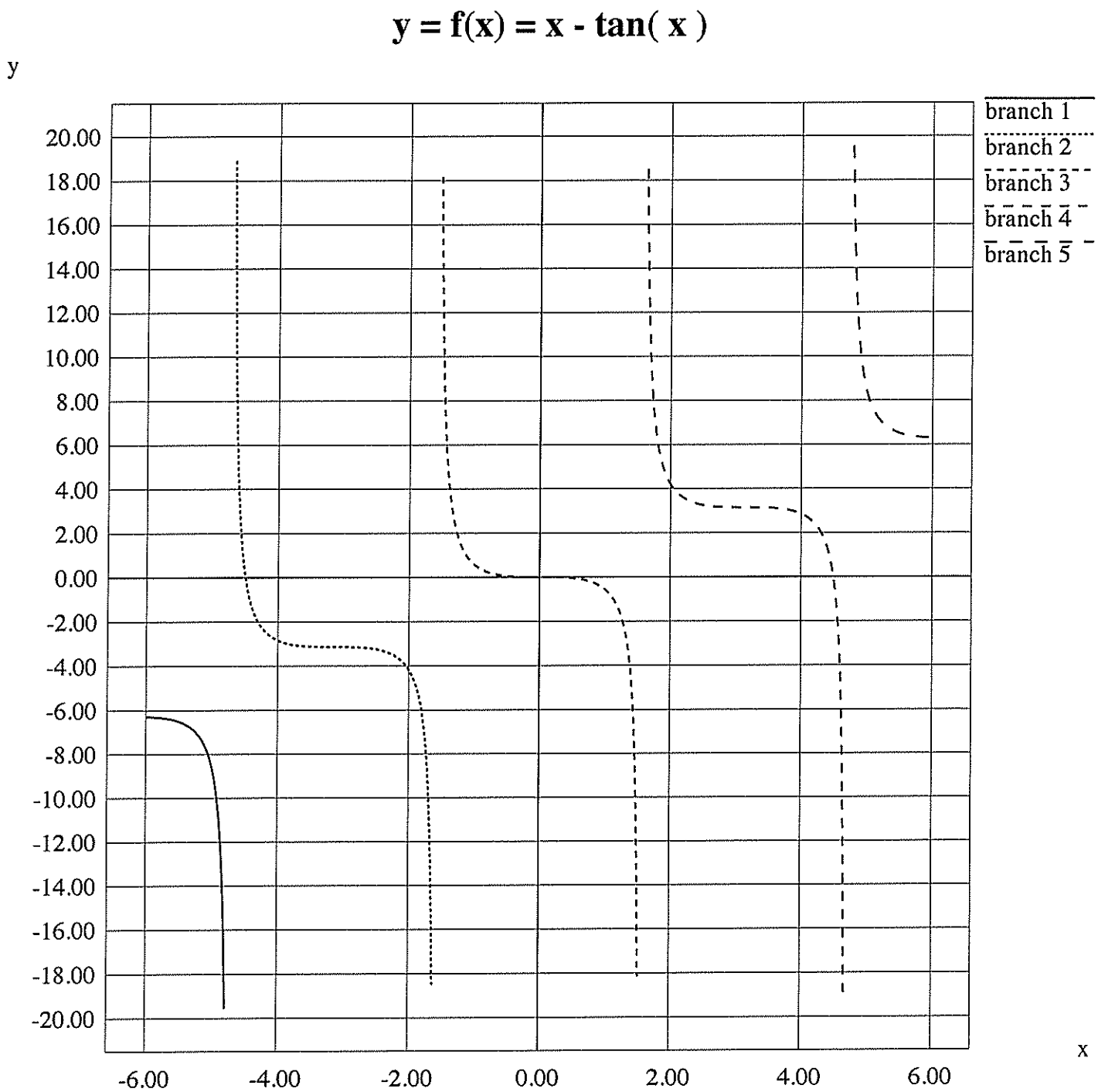
Here are some numerical results:

Start in midpoint of interval

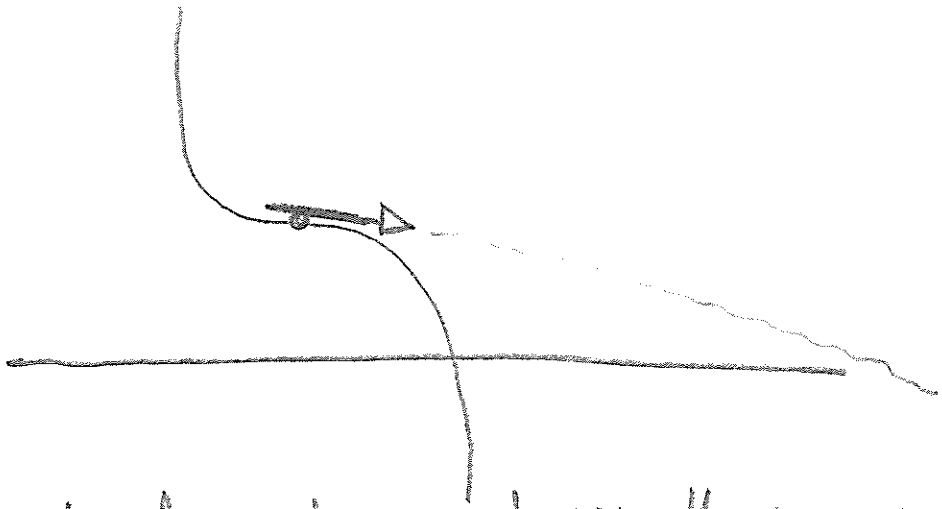
i	x_i	$f(x_i)$	$f'(x_i)$	x_{i+1}
0	4.0	2.84	-1.34	6.12
1	6.12	6.28	-0.027	238.4
2	238.4	238.7	-0.137	1978.8
3	1978.8	1979.2	-0.234	10,494

The sequence diverges!

Newton's method applied to:



See plot on handout



If you start where slope is too small, you get pushed away from the root due to $f'(x_i)$ in the denominator

We see that Newton's method does not always converge

See handout of results:

- gives table of results for different initial guesses
- convergence/divergence depends very much on the initial guess
- ⊙ note convergence to root at $x=0$ is very slow since it is a multiple root $f'(0)=0$ too.

Since Newton's method can be thought of as a fixed point method $g(x) = x - f(x)/f'(x)$ our theory can be analyzed to show convergence when $\left| \frac{f(x)\, f''(x)}{(f'(x))^2} \right| < 1$ in neighbourhood of root.

Newton's Method for $x_0 \in \{4.2, 4.3, \ldots, 4.8\}$ and also 0.1

Stop when $\Delta x_i < 5 \times 10^{-7}$ or too many iterations.

i	x_i	Δx_i
0	**4.200000E + 00**	
1	$4.966404E+00$	$7.7E-01$
2	$5.560783E+00$	$5.9E-01$
3	$1.385459E+01$	$8.3E+00$
4	$1.473222E+01$	$8.8E-01$
5	$2.215927E+01$	$7.4E+00$
6	$7.855419E+02$	$7.6E+02$
7	$3.829015E+04$	$3.8E+04$
8	$2.342654E+05$	$2.0E+05$
9	$2.115161E+12$	$2.1E+12$
10	$4.707626E+12$	$2.6E+12$

divergence

i	x_i	Δx_i
0	**4.300000E + 00**	
1	$4.685476E+00$	$3.9E-01$
2	$4.661952E+00$	$2.4E-02$
3	$4.623352E+00$	$3.9E-02$
4	$4.570926E+00$	$5.2E-02$
5	$4.521218E+00$	$5.0E-02$
6	$4.497028E+00$	$2.4E-02$
7	$4.493471E+00$	$3.6E-03$
8	$4.493409E+00$	$6.2E-05$
9	$4.493409E+00$	$1.8E-08$

Convergence

i	x_i	Δx_i
0	**4.400000E + 00**	
1	$4.535981E+00$	$1.4E-01$
2	$4.501860E+00$	$3.4E-02$
3	$4.493745E+00$	$8.1E-03$
4	$4.493410E+00$	$3.4E-04$
5	$4.493409E+00$	$5.3E-07$
6	$4.493409E+00$	$1.3E-12$

Convergence

i	x_i	Δx_i
0	**4.500000E + 00**	
1	$4.493614E+00$	$6.4E-03$
2	$4.493410E+00$	$2.0E-04$
3	$4.493409E+00$	$2.0E-07$

Convergence

i	x_i	Δx_i
0	**4.600000E + 00**	
1	$4.545732E+00$	$5.4E-02$
2	$4.506146E+00$	$4.0E-02$
3	$4.494172E+00$	$1.2E-02$
4	$4.493412E+00$	$7.6E-04$
5	$4.493409E+00$	$2.7E-06$
6	$4.493409E+00$	$3.5E-11$

Convergence

i	x_i	Δx_i
0	**4.700000E + 00**	
1	$4.688332E+00$	$1.2E-02$
2	$4.666984E+00$	$2.1E-02$
3	$4.631183E+00$	$3.6E-02$
4	$4.580473E+00$	$5.1E-02$
5	$4.528429E+00$	$5.2E-02$
6	$4.499138E+00$	$2.9E-02$
7	$4.493564E+00$	$5.6E-03$
8	$4.493410E+00$	$1.5E-04$
9	$4.493409E+00$	$1.1E-07$

Convergence

i	x_i	Δx_i
0	**4.800000E + 00**	
1	$4.924869E+00$	$1.2E-01$
2	$5.369818E+00$	$4.4E-01$
3	$9.342083E+00$	$4.0E+00$
4	$1.381284E+03$	$1.4E+03$
5	$1.910815E+03$	$5.3E+02$
6	$4.328343E+03$	$2.4E+03$
7	$8.910136E+03$	$4.6E+03$
8	$2.970514E+04$	$2.1E+04$
9	$3.074601E+04$	$1.0E+03$
10	$6.584543E+04$	$3.5E+04$

divergence

i	x_i	Δx_i
0	**1.000000E − 01**	
1	$6.675568E-02$	$3.3E-02$
2	$4.453025E-02$	$2.2E-02$
3	$2.969468E-02$	$1.5E-02$
4	$1.979878E-02$	$9.9E-03$
5	$1.319988E-02$	$6.6E-03$
6	$8.800124E-03$	$4.4E-03$
7	$5.866810E-03$	$2.9E-03$
8	$3.911224E-03$	$2.0E-03$
9	$2.607488E-03$	$1.3E-03$
10	$1.738327E-03$	$8.7E-04$
11	$1.158885E-03$	$5.8E-04$
12	$7.725903E-04$	$3.9E-04$
13	$5.150602E-04$	$2.6E-04$
14	$3.433735E-04$	$1.7E-04$
15	$2.289157E-04$	$1.1E-04$
16	$1.526104E-04$	$7.6E-05$
17	$1.017403E-04$	$5.1E-05$
18	$6.782686E-05$	$3.4E-05$
19	$4.521791E-05$	$2.3E-05$
20	$3.014527E-05$	$1.5E-05$

this iteration converges to $x=0$ where $f'(x)=0$

The convergence rate is reduced from quadratic to linear.

So what method is used in general purpose software?

idea: marry the certain convergence of bisection method with speed of secant method

since derivative not needed.

just like sorting algorithms combine approaches

- start with $[B, C]$ as bounding interval
- try secant step: D = secant step if outside of interval, better of taking Midpt of $[B, C]$ → replace D by M
- revise interval to $[B, D]$ or $[D, C]$ and repeat

- once secant starts to converge, it converges quickly.

(finer details omitted due to time)

see scipy.optimize.root_scalar

the end